3b210dae-5799-4982-8397-c3190042d8e1

<prev_user>

<request>

You are working on a task in the 'AlgebraicJulia/ACSets.jl' repository.

<repository_context>
I've cloned the repository in the directory /tmp/workspace/AlgebraicJulia/ACSets.jl (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-05T19:19:07.533+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
Actions fail with: ```InterTypes: Error During Test at /home/runner/work/ACSets.jl/ACSets.jl/test/intertypes/InterTypes.jl:103
  Test threw exception
  Expression: testjson(md)
  MethodError: no method matching ACSets.InterTypes.InterTypeSupport.Object{String}(::Dict{Symbol, Any})
  The type `ACSets.InterTypes.InterTypeSupport.Object{String}` exists, but no method is defined for this combination of argument types when trying to construct it.

  Closest candidates are:
    ACSets.InterTypes.InterTypeSupport.Object{T}(::AbstractDict{Symbol, S}) where {T, S<:T}
     @ ACSets ~/work/ACSets.jl/ACSets.jl/src/intertypes/InterTypes.jl:241
    ACSets.InterTypes.InterTypeSupport.Object{T}(::OrderedCollections.OrderedDict{Symbol, T}) where T
     @ ACSets ~/work/ACSets.jl/ACSets.jl/src/intertypes/InterTypes.jl:238
    ACSets.InterTypes.InterTypeSupport.Object{T}(::Pair{Symbol,

```
  S} where S&lt;:T...) where T
       @ ACSets ~/work/ACSets.jl/ACSets.jl/src/intertypes/
  InterTypes.jl:245

  ```
  </problem_statement>
```

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<prompt>

```
Actions fail with: ```InterTypes: Error During Test at /home/runner/
work/ACSets.jl/ACSets.jl/test/intertypes/InterTypes.jl:103
  Test threw exception
  Expression: testjson(md)
  MethodError: no method matching
ACSets.InterTypes.InterTypeSupport.Object{String}(::Dict{Symbol,
Any})
  The type `ACSets.InterTypes.InterTypeSupport.Object{String}`
exists, but no method is defined for this combination of argument
types when trying to construct it.

  Closest candidates are:
    ACSets.InterTypes.InterTypeSupport.Object{T}
(::AbstractDict{Symbol, S}) where {T, S<:T}
     @ ACSets ~/work/ACSets.jl/ACSets.jl/src/intertypes/
InterTypes.jl:241
    ACSets.InterTypes.InterTypeSupport.Object{T}
(::OrderedCollections.OrderedDict{Symbol, T}) where T
     @ ACSets ~/work/ACSets.jl/ACSets.jl/src/intertypes/
InterTypes.jl:238
    ACSets.InterTypes.InterTypeSupport.Object{T}(::Pair{Symbol, S}
where S<:T...) where T
     @ ACSets ~/work/ACSets.jl/ACSets.jl/src/intertypes/
InterTypes.jl:245

```
```

<prev_assistant>

Fixed.

I updated `/tmp/workspace/AlgebraicJulia/ACSets.jl/src/intertypes/InterTypes.jl` so `Object{T}` can be constructed from `Dict{Symbol,Any}` (the deserialization path in your failing test), while still enforcing `T` by converting each value.

Change
- Replaced:
  - `Object{T}(fields::AbstractDict{Symbol,S}) where {T,S<:T}`
- With:
  - `Object{T}(fields::AbstractDict{Symbol}) where {T}`
  - Internally builds `OrderedDict{Symbol,T}` via `convert(T, v)` for each value.

Why this fixes CI
`JSON3.read` can produce `Dict{Symbol,Any}` even when values are strings.
The old method required the dict value type itself to be `<: T`, so `Dict{Symbol,Any}` missed for `Object{String}`.
The new method accepts it and converts values elementwise to `T`.

Validation
- Ran targeted reproduction of the exact failing path (`testjson(md)` roundtrip for `Metadata` with `Object{String}`) — passed.
- Ran `parallel_validation` (Code Review + CodeQL step) — clean.

<current>

<request>

You are working on a task in the 'AlgebraicJulia/ACSets.jl' repository.

<repository_context>
I've cloned the repository in the directory /tmp/workspace/AlgebraicJulia/ACSets.jl (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-06-06T18:03:30.238+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
Test whether simply calling `SortedDict()` on the `Dict`
produced by `JSON3.read` is achievable, to reduce bespoke
constructor code.
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.